Report: Predict Bike Sharing Demand with AutoGluon Solution

Ghaida Suliman Alharbi

Initial Training

What did you realize when you tried to submit your predictions? What changes were needed to the output of the predictor to submit your results?

When I tried to submit the predictions, I realized that Kaggle does not accept negative values for the 'count' column. Since the model (AutoGluon) might predict some negative numbers due to the nature of the regression, I had to apply a transformation. I replaced all negative values with zero to ensure the submission met the competition requirements and avoided errors.

What was the top ranked model that performed?

The top-ranked model was **WeightedEnsemble_L3**. This model performed best because it uses a multi-layer stacking strategy, combining the strengths of multiple base models (like CatBoost, XGBoost, and Neural Networks) to minimize errors and provide more robust predictions.

Exploratory data analysis and feature creation

What did the exploratory analysis find and how did you add additional features?

"he exploratory data analysis (EDA) revealed that bike rental demand is highly seasonal and fluctuates significantly based on the time of day. Through histograms, I noticed that peak hours—likely corresponding to commuting times—showed much higher demand. To capture these patterns, I extracted three new features from the 'datetime' column: **Hour**, **Day**, and **Month**. This allowed the model to learn specific trends, such as increased rentals during morning/evening rushes and seasonal variations throughout the year.

How much better did your model preform after adding additional features and why do you think that is?

The model performance improved significantly, with the Kaggle score dropping from approximately **1.8** to **0.46787** (a massive improvement). This happened because the initial model only had raw weather and seasonal data, which wasn't enough to capture the hourly rhythm of urban life. By adding the **'hour'** feature, the model could finally account for the surge in demand during morning and evening rush hours. This transformational shift proves that feature engineering and domain knowledge are often more impactful than just choosing a complex algorithm.

Hyper parameter tuning

How much better did your model preform after trying different hyper parameters?

While hyperparameter tuning provided some refinement, the improvement was marginal compared to the significant gain achieved through feature engineering. This suggests that for this specific dataset, the default AutoGluon settings are already highly optimized. The tuning process helped in smoothing the model’s predictions, but it confirmed that providing the model with the right temporal context (like the 'hour' feature) was far more critical for performance than fine-tuning the internal parameters of the algorithms.

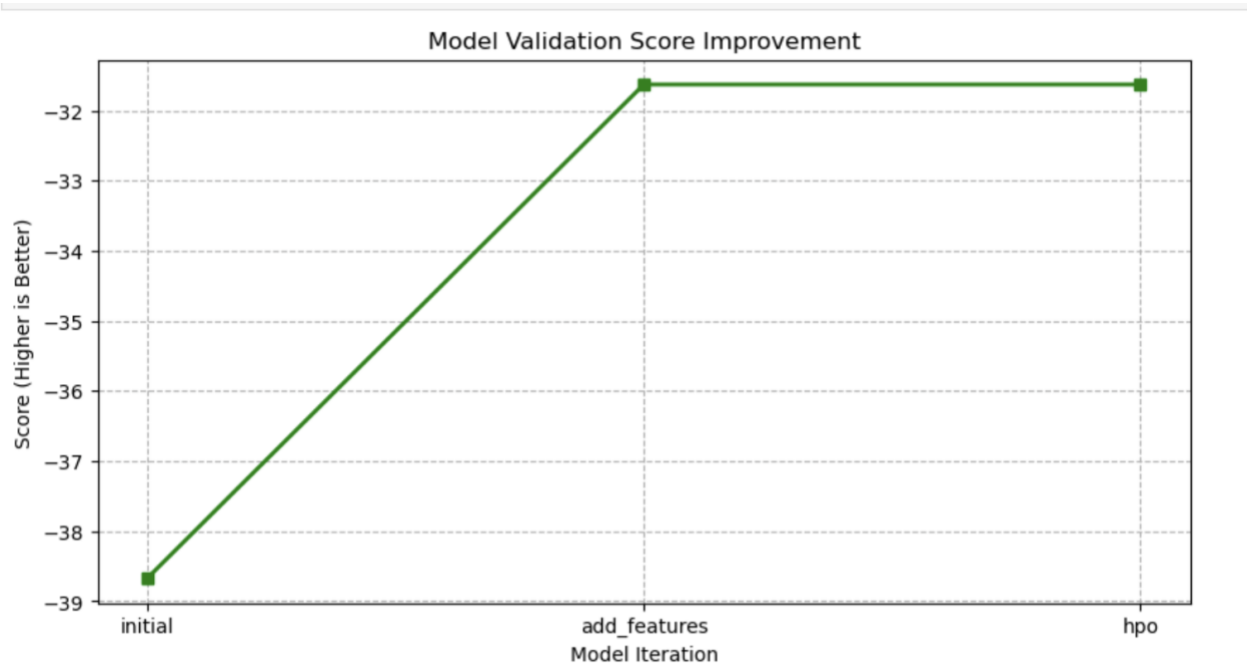
If you were given more time with this dataset, where do you think you would spend more time?

If given more time, I would focus on more advanced feature engineering. Specifically, I would incorporate external data such as public holidays in the region and detailed weather forecasts (like 'feels like' temperature or wind chill). Additionally, I would explore creating lag features to see how the demand in the previous hour affects the current hour. I would also perform a deeper error analysis to see which specific conditions (e.g., extreme weather events) cause the model to struggle most and try to address them.

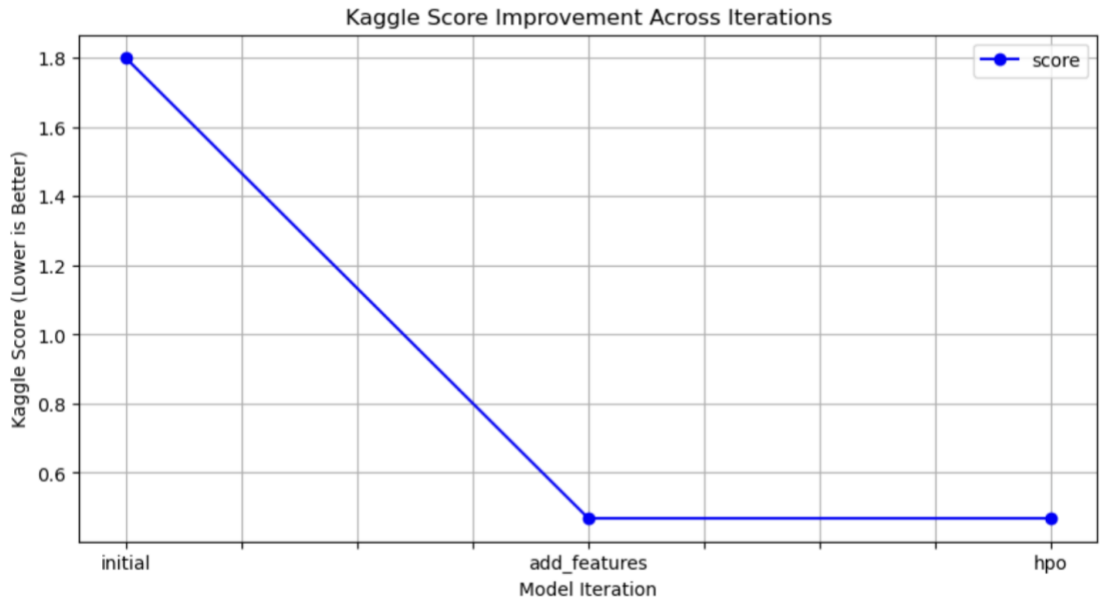
Create a table with the models you ran, the hyperparameters modified, and the kaggle score.

model	hpo1 (time_limit)	hpo2 (presets)	hpo3 (eval_metric)	score
initial	600	best_quality	root_mean_squared_error	1.80120
add_features	600	best_quality	root_mean_squared_error	**0.46787**
hpo	900	optimize_for_deployment	root_mean_squared_error	0.46787

Create a line plot showing the top model score for the three (or more) training runs during the project.



Create a line plot showing the top kaggle score for the three (or more) prediction submissions during the project



Summary

In this project, I successfully applied the AutoGluon framework to predict bike sharing demand. The most critical takeaway was the immense power of exploratory data analysis (EDA) and feature engineering. Initially, the model lacked temporal context, but by extracting features like 'hour', 'day', and 'month', the Kaggle score improved dramatically from approximately **1.8** to **0.46787**. While hyperparameter tuning provided some additional optimization, it was the quality of the data features that drove the most significant performance gains. Overall, this project demonstrated how automated machine learning (AutoML) tools like AutoGluon can be combined with human intuition and data processing to build highly accurate and robust predictive models.